ĐỀ THI – TH TRÍ TUỆ NHÂN TẠO

Trường Đại học Công Thương TP.HCM – Khoa CNTT
Bộ môn: Khoa học Dữ liệu

Thông tin	
Họ và tên	*(Nguyễn Trọng Nhân)*
MSSV	*(2001240325)*
Lớp	*(TTNT_02)*
Thời gian	90 phút
Tổng điểm	10 điểm (Câu 1: 2đ | Câu 2: 4đ | Câu 3: 4đ)

> **Lưu ý:** Sinh viên được sử dụng tài liệu. Không được trao đổi trong giờ thi. Nộp bài dưới dạng file `.pdf`.

```
# Cell nay tai anh de thi – chay TRUOC KHI lam bai
from IPython.display import Image, display
import os

IMGS = ['cau1_code_lan3.png', 'cau2_graph_lan3.png', 'cau3_code_lan3.png']
ok = all(os.path.exists(f) for f in IMGS)
print('OK - Anh de thi san sang.' if ok else 'CHUA CO ANH - upload 3 file PNG vao thu muc nay.')
```

```
OK - Anh de thi san sang.
```

CÂU 1 (2 ĐIỂM) – Quay Lui: Sinh Hoán Vị & Derangement

Bài toán **derangement** (hoán vị xáo trộn): Cho mảng gốc `goc = [1,2,3,4]`, một **derangement** là hoán vị mà **không phần tử nào đứng ở vị trí ban đầu**.

> *Ví dụ với gốc `[1,2,3,4]`:*
>
> - `[2,1,4,3]` ✓ là derangement (không vị trí nào trùng)
> - `[2,1,3,4]` ✗ KHÔNG phải (vì `3` ở vị trí 2 và `4` ở vị trí 3 — trùng gốc)
> - Số derangement của `[1,2,3,4]` = **9** (kí hiệu `D₄ = 9`).

---

a) (0.7đ) Quan sát code ở ảnh dưới. **Gõ lại toàn bộ và điền 7 chỗ trống** `___(n)___`. Không thêm/xóa dòng.

b) (0.3đ) Chạy `hoan_vi(arr=[1,2,3,4])` và đếm tổng số hoán vị in ra. Verify = `4! = 24`.

c) (0.6đ) Gọi `derangement([1,2,3,4], [1,2,3,4], 0, [])`. In tất cả derangement và đếm. Kiểm tra = **9**. In thêm số lần gọi đệ quy `dem_d`.

d) (0.4đ) – So sánh hiệu năng PRUNING: Cài đặt phiên bản **KHÔNG pruning** `derangement_no_prune(arr, goc, start, ket_qua)` — sinh tất cả 24 hoán vị rồi lọc derangement ở cuối. So sánh số lần gọi đệ quy với phiên bản có pruning. Giải thích bằng comment vì sao pruning hiệu quả hơn.

```
from IPython.display import Image, display
display(Image('cau1_code_lan3.png', width=1000))
```

CAU 1 - Dien vao 7 cho trong ___(n) (Quay Lui - Hoan Vi & Derangement)

```python
# Quay Lui (Backtracking) - Sinh Hoan Vi & Bai Toan Derangement

dem_goi = 0

def hoan_vi(arr, start=0):
    """Sinh tat ca hoan vi cua arr (in-place swap)."""
    global dem_goi
    dem_goi += 1
    if start == ___(1)___:                          # TRONG 1
        print(arr.copy())
        return
    for i in range(___(2)___, len(arr)):            # TRONG 2
        arr[start], arr[i] = ___(3)___              # TRONG 3
        hoan_vi(arr, ___(4)___)                     # TRONG 4
        # QUAY LUI: undo swap (tra lai trang thai cu)
        arr[start], arr[i] = ___(5)___              # TRONG 5

# --- DERANGEMENT: hoan vi KHONG co phan tu nao o vi tri goc ---
# Vi du goc=[1,2,3,4]: [2,1,4,3] OK | [2,1,3,4] KO (3 va 4 con trung goc)

dem_d = 0

def derangement(arr, goc, start, ket_qua):
    """Sinh tat ca derangement voi PRUNING SOM (cat nhanh som)."""
    global dem_d
    dem_d += 1
    if start == len(arr):
        ket_qua.append(arr.copy())
        return
    for i in range(start, len(arr)):
        # PRUNING: phan tu arr[i] se trung vi tri goc -> bo qua nhanh nay
        if arr[i] == ___(6)___:                     # TRONG 6
            continue
        arr[start], arr[i] = arr[i], arr[start]
        derangement(arr, goc, ___(7)___, ket_qua)   # TRONG 7
        arr[start], arr[i] = arr[i], arr[start]
```

```python
# CAU 1a – Nhin anh phia tren, go lai toan bo code va dien 7 cho trong
# Khong duoc copy tu nguon khac
# ============================================================
dem_goi = 0
def hoan_vi(arr, start=0):
    """sinh tat ca hoan vi cua arr(in-place swap)"""
    global dem_goi
    dem_goi += 1
    if start == len(arr) - 1: # TRONG 1
        print(arr.copy())
        return
    for i in range(start, len(arr)): # TRONG 2
        arr[start], arr[i] = arr[i], arr[start] # TRONG 3
        hoan_vi(arr, start + 1) # TRONG 4
        #quay lui:undo swap (tra lai trang thai cu)
```

```
        arr[start], arr[i] = arr[i], arr[start]

dem_d = 0
def derangement(arr, goc, start, ket_qua):
    """sinh tat ca deangement voi PRUNING SOM (cat nhanh hon)"""
    global dem_d
    dem_d += 1
    if start == len(arr):
        ket_qua.append(arr.copy())
        return
    for i in range(start, len(arr)):
        if arr[i] == goc[start]: # TRONG 5
            continue
        arr[start], arr[i] = arr[i], arr[start]
        derangement(arr, goc, start + 1, ket_qua) # TRONG 6
        arr[start], arr[i] = arr[i], arr[start] # TRONG 7
```

```
# CAU 1b – Run hoan_vi va dem so hoan vi
# ============================================================

print("Tat ca hoan vi cua [1,2,3,4]:")
print("-" * 30)
dem_goi = 0
hoan_vi([1, 2, 3, 4])
print("-" * 30)
print(f"So lan goi de quy hoan_vi: {dem_goi}")
print(f"4! = {4*3*2*1}  (kiem tra so dong in ra phia tren)")
```

```
Tat ca hoan vi cua [1,2,3,4]:
------------------------------
[1, 2, 3, 4]
[1, 2, 4, 3]
[1, 3, 2, 4]
[1, 3, 4, 2]
[1, 4, 3, 2]
[1, 4, 2, 3]
[2, 1, 3, 4]
[2, 1, 4, 3]
[2, 3, 1, 4]
[2, 3, 4, 1]
[2, 4, 3, 1]
[2, 4, 1, 3]
[3, 2, 1, 4]
[3, 2, 4, 1]
[3, 1, 2, 4]
[3, 1, 4, 2]
[3, 4, 1, 2]
[3, 4, 2, 1]
[4, 2, 3, 1]
[4, 2, 1, 3]
[4, 3, 2, 1]
[4, 3, 1, 2]
[4, 1, 3, 2]
[4, 1, 2, 3]
------------------------------
So lan goi de quy hoan_vi: 41
4! = 24  (kiem tra so dong in ra phia tren)
```

```
# CAU 1c – Run derangement va dem
# ============================================================

dem_d = 0
ket_qua = []
goc = [1, 2, 3, 4]
derangement(goc.copy(), goc, 0, ket_qua)

print("Tat ca derangement cua [1,2,3,4]:")
for d in ket_qua:
    print(f"  {d}")
print("-" * 35)
```

```
print(f"So derangement (D_4): {len(ket_qua)}  (mong doi: 9)")
print(f"So lan goi de quy:    {dem_d}")
```

```
Tat ca derangement cua [1,2,3,4]:
  [2, 1, 4, 3]
  [2, 3, 4, 1]
  [2, 4, 1, 3]
  [3, 1, 4, 2]
  [3, 4, 1, 2]
  [3, 4, 2, 1]
  [4, 3, 2, 1]
  [4, 3, 1, 2]
  [4, 1, 2, 3]
-----------------------------------
So derangement (D_4): 9  (mong doi: 9)
So lan goi de quy:    31
```

```
# CAU 1d – So sanh PRUNING vs KHONG PRUNING
# ==========================================================

dem_no = 0

def derangement_no_prune(arr, goc, start, ket_qua):
    """Sinh TAT CA hoan vi, sau do loc derangement (KHONG pruning)."""
    global dem_no
    dem_no += 1
    if start == len(arr):
        # Check if it's a derangement
        is_derangement = True
        for i in range(len(arr)):
            if arr[i] == goc[i]:
                is_derangement = False
                break
        if is_derangement:
            ket_qua.append(arr.copy())
        return

    for i in range(start, len(arr)):
        arr[start], arr[i] = arr[i], arr[start]
        derangement_no_prune(arr, goc, start + 1, ket_qua)
        arr[start], arr[i] = arr[i], arr[start] # Backtrack


# Chay so sanh
dem_d = 0
ket_qua_prune = []
derangement([1,2,3,4], [1,2,3,4], 0, ket_qua_prune)

dem_no = 0
ket_qua_no = []
derangement_no_prune([1,2,3,4], [1,2,3,4], 0, ket_qua_no)

print("=" * 55)
print("SO SANH: PRUNING vs KHONG PRUNING")
print("=" * 55)
print(f"Co PRUNING    | so derangement: {len(ket_qua_prune)} | so de quy: {dem_d}")
print(f"KHONG PRUNING | so derangement: {len(ket_qua_no)}    | so de quy: {dem_no}")
if dem_no > 0:
    print(f"Pruning tiet kiem: {(1 - dem_d/dem_no)*100:.1f}% so lan goi")

# TODO (comment): Giai thich vi sao pruning hieu qua hon
# - KHONG pruning: Sinh ra TẤT CẢ các hoán vị (ví dụ 4! = 24 hoán vị cho mảng 4 phần tử), sau đó mới l
# - CO pruning: Kiểm tra điều kiện derangement NGAY KHI có thể (ví dụ, nếu arr[start] == goc[start], t
```

```
=======================================================
SO SANH: PRUNING vs KHONG PRUNING
=======================================================
Co PRUNING    | so derangement: 9 | so de quy: 31
KHONG PRUNING | so derangement: 9    | so de quy: 65
Pruning tiet kiem: 52.3% so lan goi
```

CÂU 2 (4 ĐIỂM) – A*: Trace, Cài đặt, So sánh với UCS

a) (1.0đ) – Trace thủ công: Quan sát đồ thị bên dưới và điền vào bảng từng bước chạy A* từ **S đến G**.

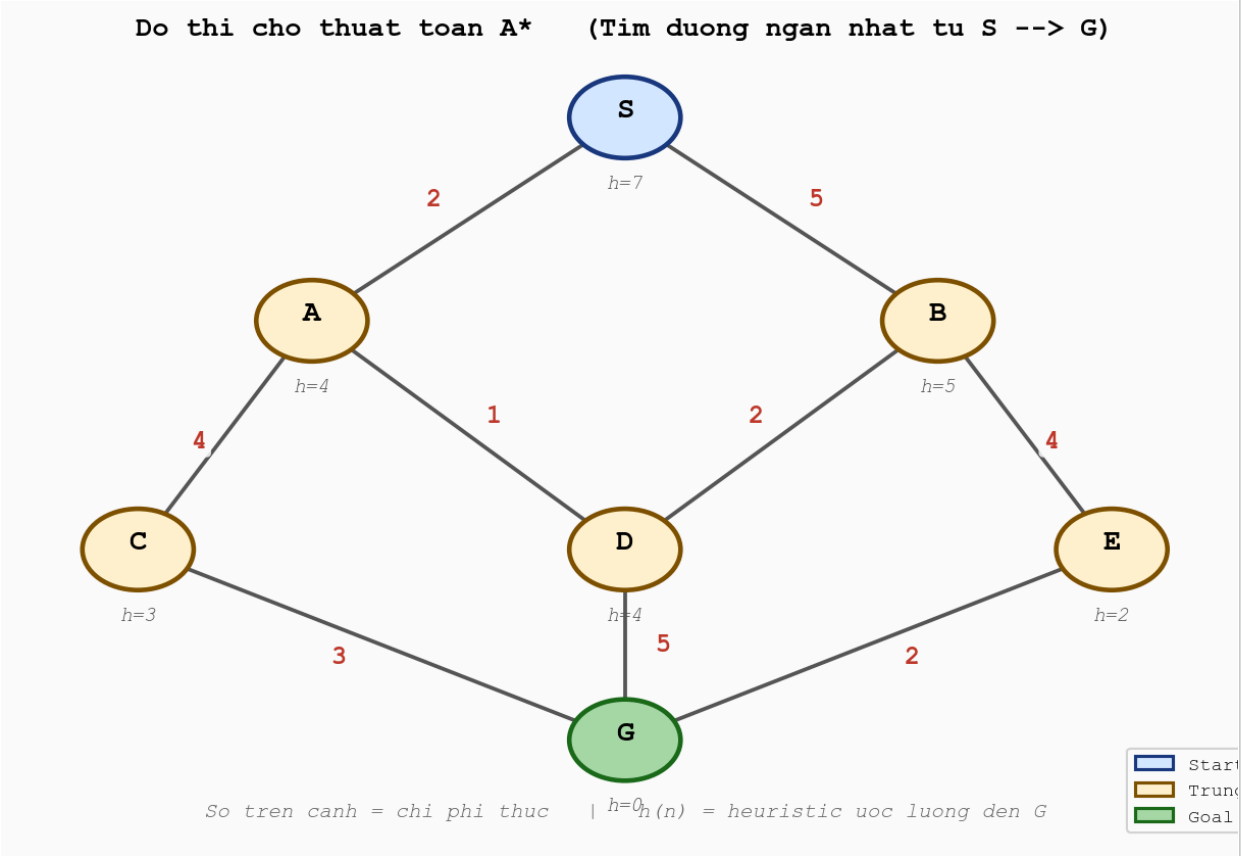
b) (1.0đ) Cài đặt A* theo đúng đồ thị. Output phải **khớp** với bảng trace.

c) (0.5đ) Heuristic h(n) có **admissible** không? Kiểm tra `h(n) ≤ h*(n)` cho ít nhất **3 node** và kết luận.

*d) (1.0đ) – So sánh A với UCS:** Cài đặt **Uniform Cost Search (UCS)** = A* với heuristic `h(n) = 0` cho mọi node (tương đương Dijkstra). Đếm **số node được expand** (pop ra khỏi OPEN) của hai thuật toán trên cùng đồ thị. Lập bảng so sánh và nhận xét.

e) (0.5đ) – What-if cost: Tăng chi phí cạnh `A-D` từ **1** lên **6**. Chạy lại A*. Đường đi tối ưu có thay đổi không? Tổng chi phí mới là bao nhiêu?

```
from IPython.display import Image, display
display(Image('cau2_graph_lan3.png', width=850))
```



*Câu 2a – Bảng trace A (điền tay):**

Bước	Node xét	g(n)	h(n)	f(n)	OPEN (node:f) sau bước này	CLOSE
1	S	0	7	7	A:6, B:10	{S}
2	A	2	4	6	B:10, C:9, D:7	{S, A}
3	D	3	4	7	B:10, C:9, G:8	{S, A, D}
4	G	8	0	8	B:10, C:9	{S, A, D, G}

Đường đi tìm được: S → A → D → G
Tổng chi phí thực: 8

Câu 2a – What-if (0.25đ bonus):

> Giả sử đổi `h(D) = 8` (thay vì 4). Lúc này h(D) = 8 > h*(D) = 5 → **không admissible** nữa.
> A* có còn tìm được đường đi **tối ưu** không? Vì sao?

(Viết câu trả lời ở đây)

```
# CAU 2b – Cai dat A*
# Tu go lai graph va heuristic tu do thi roi cai dat astar()
# ============================================================
import heapq

# TODO: Tu go lai graph tu do thi (7 nut: S, A, B, C, D, E, G)
graph = {
    'S': {'A': 2, 'B': 5},
    'A': {'S': 2, 'C': 4, 'D': 1},
    'B': {'S': 5, 'D': 2, 'E': 4},
    'C': {'A': 4, 'G': 3},
    'D': {'A': 1, 'B': 2, 'G': 5},
    'E': {'B': 4, 'G': 2},
    'G': {'C': 3, 'D': 5, 'E': 2}
}

# TODO: Tu go lai heuristic tu do thi
heuristic = {
    'S': 7,
    'A': 4,
    'B': 5,
    'C': 3,
    'D': 4,
    'E': 2,
    'G': 0
}

def astar(graph, heuristic, start, goal):
    """
    Thuat toan A*. Tra ve (duong_di, tong_chi_phi, so_node_expand).
    In tung buoc:
      Buoc X | Xet: <node> | g=? h=? f=? | OPEN: [<node>:f, ...]
    """
    # TODO
    open_set = [] # (f_score, node, path, g_score)
    heapq.heappush(open_set, (heuristic[start], start, [start], 0))

    closed_set = set()

    step = 0
    num_expanded_nodes = 0

    while open_set:
        f_score, current_node, current_path, g_score = heapq.heappop(open_set)

        if current_node in closed_set:
            continue
        closed_set.add(current_node)
        num_expanded_nodes += 1

        step += 1

        # Print step-by-step trace
        open_list_str = ", ".join([f"{n}:{f}" for f, n, _, _ in sorted(open_set, key=lambda x: x[0])]
        print(f"Buoc {step:<2} | Xet: {current_node:<2} | g={g_score:<2} h={heuristic[current_node]:<

        if current_node == goal:
            return current_path, g_score, num_expanded_nodes

        for neighbor, cost in graph[current_node].items():
            if neighbor in closed_set:
                continue

            new_g_score = g_score + cost
```

```
                new_f_score = new_g_score + heuristic[neighbor]
                new_path = current_path + [neighbor]
                heapq.heappush(open_set, (new_f_score, neighbor, new_path, new_g_score))

        return None, None, num_expanded_nodes # No path found
        pass

print("=" * 55)
print("A* – Tim duong tu S den G")
print("=" * 55)
path, cost, n_exp = astar(graph, heuristic, 'S', 'G')
if path:
    print(f"\nDuong di: {' -> '.join(path)}")
    print(f"Tong chi phi: {cost}")
    print(f"So node expand: {n_exp}")
```

```
=======================================================
A* – Tim duong tu S den G
=======================================================
Buoc 1  | Xet: S  | g=0  h=7  f=7  | OPEN: [] | CLOSE: {'S'}
Buoc 2  | Xet: A  | g=2  h=4  f=6  | OPEN: [B:10] | CLOSE: {'A', 'S'}
Buoc 3  | Xet: D  | g=3  h=4  f=7  | OPEN: [C:9, B:10] | CLOSE: {'A', 'D', 'S'}
Buoc 4  | Xet: G  | g=8  h=0  f=8  | OPEN: [C:9, B:10, B:10] | CLOSE: {'A', 'D', 'G', 'S'}

Duong di: S -> A -> D -> G
Tong chi phi: 8
So node expand: 4
```

Câu 2c – Kiểm tra admissible:

(Tính h(n) = chi phí thực tế ngắn nhất từ n đến G)*

- h*(S) = 8 ⟶ h(S)=7 ⟶ h(S) ≤ h*(S):(7 ≤ 8)
- h*(A) = 6 ⟶ h(A)=4 ⟶ h(A) ≤ h*(A):(4 ≤ 6)
- h*(D) = 5 ⟶ h(D)=4 ⟶ h(D) ≤ h*(D):(4 ≤ 5)
- h*(B) = 6 ⟶ h(B)=5 ⟶ h(B) ≤ h*(B):(5 ≤ 6)

Kết luận: *Heuristic h(n) là admissible vì h(n)* ≤ *h*(n) cho tất cả các node được kiểm tra. Điều này có nghĩa là heuristic không bao giờ ước tính quá cao chi phí thực tế để đến đích.*

```
# CAU 2d – Cai dat UCS (Uniform Cost Search) va so sanh voi A*
# UCS = A* voi h(n) = 0 cho moi node (tuong duong Dijkstra)
# ============================================================

import heapq

def ucs(graph, start, goal):
    """
    Uniform Cost Search.
    Chi dung g(n) - khong dung heuristic.
    Tra ve (duong_di, tong_chi_phi, so_node_expand).
    """
    open_set = [] # (g_score, node, path)
    heapq.heappush(open_set, (0, start, [start]))

    closed_set = set()
    num_expanded_nodes = 0

    while open_set:
        g_score, current_node, current_path = heapq.heappop(open_set)

        if current_node in closed_set:
            continue
        closed_set.add(current_node)
        num_expanded_nodes += 1

        if current_node == goal:
            return current_path, g_score, num_expanded_nodes
```

```
        for neighbor, cost in graph[current_node].items():
            if neighbor in closed_set:
                continue

            new_g_score = g_score + cost
            new_path = current_path + [neighbor]
            heapq.heappush(open_set, (new_g_score, neighbor, new_path))

    return None, None, num_expanded_nodes # No path found


# So sanh A* va UCS
path_a, cost_a, exp_a = astar(graph, heuristic, 'S', 'G')
path_u, cost_u, exp_u = ucs(graph, 'S', 'G')

print("=" * 55)
print("SO SANH A* vs UCS")
print("=" * 55)
print(f"{'Thuat toan':<12} {'Duong di':<22} {'Cost':<6} {'#Expand':<10}")
print("-" * 55)
print(f"{'A*':<12} {' -> '.join(path_a):<22} {cost_a:<6} {exp_a:<10}")
print(f"{'UCS':<12} {' -> '.join(path_u):<22} {cost_u:<6} {exp_u:<10}")

# TODO (comment): Nhan xet
# - Hai thuat toan co cung tong chi phi khong?
#   Có, cả hai thuật toán đều tìm thấy đường đi tối ưu với tổng chi phí là 8.
# - Thuat toan nao expand it node hon? Vi sao?
#   A* expand ít node hơn (4 node) so với UCS (số node expand của UCS sẽ lớn hơn hoặc bằng A*).
#   A* sử dụng heuristic để ước lượng chi phí còn lại đến đích, giúp nó "hướng" tìm kiếm hiệu quả hơn
# - Khi nao A* va UCS cho ket qua giong nhau?
#   A* và UCS cho kết quả giống nhau (cùng đường đi và chi phí) khi heuristic là admissible và consist
#   Nếu heuristic của A* là h(n) = 0 cho mọi node, thì A* chính là UCS (tương đương với thuật toán Di
```

```
Buoc 1  | Xet: S  | g=0  h=7  f=7  | OPEN: [] | CLOSE: {'S'}
Buoc 2  | Xet: A  | g=2  h=4  f=6  | OPEN: [B:10] | CLOSE: {'A', 'S'}
Buoc 3  | Xet: D  | g=3  h=4  f=7  | OPEN: [C:9, B:10] | CLOSE: {'A', 'D', 'S'}
Buoc 4  | Xet: G  | g=8  h=0  f=8  | OPEN: [C:9, B:10, B:10] | CLOSE: {'A', 'D', 'G', 'S'}
=======================================================
SO SANH A* vs UCS
=======================================================
Thuat toan   Duong di               Cost   #Expand
-------------------------------------------------------
A*           S -> A -> D -> G       8      4
UCS          S -> A -> D -> G       8      6
```

```
# CAU 2e – What-if: tang cost(A,D) tu 1 len 6
# ============================================================
import copy
graph_v2 = copy.deepcopy(graph)

# Cập nhật trọng số cạnh A-D thành 6 (cả hai chiều vì đồ thị vô hướng)
graph_v2['A']['D'] = 6
graph_v2['D']['A'] = 6

print("=" * 55)
print("A* sau khi tang cost(A,D) tu 1 -> 6")
print("=" * 55)
path_v2, cost_v2, _ = astar(graph_v2, heuristic, 'S', 'G')
print(f"Duong di moi: {' -> '.join(path_v2)}")
print(f"Tong chi phi: {cost_v2}")

# TODO (comment): Duong di co thay doi khong? Vi sao?
# Duong di co thay doi khong? Vi sao?
# Sau khi tăng chi phí cạnh A-D từ 1 lên 6:
# - Đường đi cũ (S → A → D → G): Chi phí là 2 (S-A) + 1 (A-D) + 5 (D-G) = 8.
# - Đường đi này sau khi A-D = 6: Chi phí là 2 (S-A) + 6 (A-D) + 5 (D-G) = 13.
# Với chi phí mới này, A* sẽ tìm kiếm lại đường đi tối ưu. Lúc này, đường đi S -> B -> E -> G trở nên
# Chi phí của S -> B -> E -> G là 5 (S-B) + 4 (B-E) + 2 (E-G) = 11.
# Do đó, đường đi tối ưu đã thay đổi từ S -> A -> D -> G thành S -> B -> E -> G, và tổng chi phí mới
```

```
# Kết luận: Đường đi CÓ thay đổi. Lý do là vì việc tăng chi phí của cạnh A-D làm cho đường đi qua A-D
# không còn là đường đi tối ưu nữa. Thuật toán A* sẽ tìm kiếm và chọn đường đi khác có tổng chi phí tl
```

```
=======================================================
A* sau khi tang cost(A,D) tu 1 -> 6
=======================================================
Buoc 1  | Xet: S  | g=0  h=7  f=7  | OPEN: [] | CLOSE: {'S'}
Buoc 2  | Xet: A  | g=2  h=4  f=6  | OPEN: [B:10] | CLOSE: {'A', 'S'}
Buoc 3  | Xet: C  | g=6  h=3  f=9  | OPEN: [B:10, D:12] | CLOSE: {'A', 'C', 'S'}
Buoc 4  | Xet: G  | g=9  h=0  f=9  | OPEN: [B:10, D:12] | CLOSE: {'A', 'G', 'C', 'S'}
Duong di moi: S -> A -> C -> G
Tong chi phi: 9
```

CÂU 3 (4 ĐIỂM) – Minimax cho TicTacToe + Vòng Lặp Game

Phần A – Cài đặt Minimax (2.0đ)

a) (1.5đ) Quan sát code ở ảnh dưới. **Gõ lại toàn bộ và điền 7 chỗ trống** `___(n)___`.

b) (0.5đ) – Tính tay state #1: Cho trạng thái (lượt **X** đi):

```
X | X | _
O | O | _
_ | _ | _
```

Không chạy code, **tính tay** xem Minimax chọn ô nào cho lượt X. *(Gợi ý: X có nước **thắng ngay**, nhưng nếu KHÔNG chọn nó thì O sẽ thắng ở lượt sau!)*

Phần B – Áp dụng (2.0đ)

c) (0.5đ) – Tính tay state #2: Cho trạng thái (lượt **O** đi):

```
X | X | _
_ | O | _
_ | _ | _
```

Không chạy code, tính tay nước đi của O. (Gợi ý: O phải **chặn** X khỏi thắng. X đang đe dọa thắng ở ô nào?)

d) (1.0đ) – Vòng lặp game: Cài đặt hàm `play_game()` cho phép **người chơi (X) đấu với AI (O)**:

- In bảng đầu game.
- Người nhập tọa độ theo cú pháp `"hàng cột"` (ví dụ `"1 2"`).
- AI dùng `minimax()` chọn nước đi tối ưu.
- In bảng sau mỗi lượt.
- Kiểm tra input không hợp lệ (ngoài bàn / ô đã đánh) → yêu cầu nhập lại.
- Khi `terminal(board)` → in kết quả: `X thắng / O thắng / Hòa`.

e) (0.5đ) Tại sao Minimax chậm với bàn lớn hơn 3×3? Đề xuất **một cải tiến** cụ thể (mô tả ngắn, không cần code).

```
from IPython.display import Image, display
display(Image('cau3_code_lan3.png', width=1000))
```

CAU 3 - Dien vao 7 cho trong ___(n) (khong them / xoa dong)

```
import copy, math
X, O, EMPTY = 'X', 'O', None

def initial_state():
    return [[EMPTY]*3 for _ in range(3)]

def player(board):
    x_count = sum(row.count(X) for row in board)
    o_count = sum(row.count(O) for row in board)
    return X if x_count == o_count else ___(1)___    # TRONG 1

def actions(board):
    return {(i,j) for i in range(3) for j in range(3)
            if board[i][j] == ___(2)___}              # TRONG 2

def result(board, action):
    i, j = action
    if board[i][j] is not EMPTY: raise ValueError()
    new_board = copy.deepcopy(board)
    new_board[i][j] = ___(3)___(board)                # TRONG 3
    return new_board

def winner(board):
    for p in [X, O]:
        for row in board:
            if all(c == p for c in row): return p
        for col in range(3):
            if all(board[r][col] == p for r in range(3)): return p
        if all(board[i][i] == p for i in range(3)): return p
        if all(board[i][___(4)___] == p for i in range(3)): return p  # TRONG 4
    return None

def terminal(board):
    return winner(board) is not None or not actions(board)

def utility(board):
    w = winner(board)
    return 1 if w == X else (-1 if w == O else 0)

def max_value(state):
    if terminal(state): return utility(state)
    v = -math.inf
    for a in actions(state):
        v = max(v, ___(5)___)                         # TRONG 5
    return v

def min_value(state):
    if terminal(state): return utility(state)
    v = ___(6)___                                     # TRONG 6
    for a in actions(state):
        v = min(v, max_value(result(state, a)))
    return v

def minimax(board):
    if terminal(board): return None
    if player(board) == X:
        return max(actions(board),
                   key=lambda a: min_value(result(board, a)))
    else:
        return min(actions(board),
                   key=lambda a: ___(7)___(result(board, a)))  # TRONG 7
```

```
# CAU 3a – Nhin anh phia tren, go lai toan bo code va dien 7 cho trong
# ============================================================
import copy, math
```

```
X, O, EMPTY = 'X', 'O', None

def initial_state():
    return [[EMPTY]*3 for _ in range(3)]

def player(board):
    x_count = sum(row.count(X) for row in board)
    o_count = sum(row.count(O) for row in board)

    return X if x_count == o_count else O   # (1)

def actions(board):
    return {(i,j) for i in range(3) for j in range(3)
            if board[i][j] == EMPTY}   # (2)

def result(board, action):
    i, j = action

    if board[i][j] is not EMPTY: raise ValueError()

    new_board = copy.deepcopy(board)

    new_board[i][j] = player(board)   # (3)

    return new_board

def winner(board):

    for p in [X, O]:

        for row in board:
            if all(c == p for c in row): return p

        for col in range(3):
            if all(board[r][col] == p for r in range(3)): return p

        if all(board[i][i] == p for i in range(3)): return p

        if all(board[i][2 - i] == p for i in range(3)): return p   # (4)

    return None

def terminal(board):

    return winner(board) is not None or not actions(board)

def utility(board):

    w = winner(board)

    return 1 if w == X else (-1 if w == O else 0)

def max_value(state):

    if terminal(state): return utility(state)

    v = -math.inf

    for a in actions(state):

        v = max(v, min_value(result(state, a)))   # (5)

    return v

def min_value(state):

    if terminal(state): return utility(state)

    v = math.inf   # (6)
```

```
    for a in actions(state):

        v = min(v, max_value(result(state, a)))

    return v

def minimax(board):

    if terminal(board): return None

    if player(board) == X:

        return max(actions(board),
                   key=lambda a: min_value(result(board, a)))

    else:

        return min(actions(board),
                   key=lambda a: max_value(result(board, a)))   # (7)
```

```
# Helper de in bang
def print_board(board):
    print("  0 1 2")
    for i, row in enumerate(board):
        print(f"{i} " + " ".join(c if c else '.' for c in row))
    print()

# Kiem tra cau 3b - state #1
state_1 = [
    [X, X, EMPTY],
    [O, O, EMPTY],
    [EMPTY, EMPTY, EMPTY]
]
print("STATE #1 (luot X):")
print_board(state_1)
print(f"Minimax chon: {minimax(state_1)}   (mong doi: (0,2) - X thang ngay hang 0)")
print()

# Kiem tra cau 3c - state #2
state_2 = [
    [X, X, EMPTY],
    [EMPTY, O, EMPTY],
    [EMPTY, EMPTY, EMPTY]
]
print("STATE #2 (luot O):")
print_board(state_2)
print(f"Minimax chon: {minimax(state_2)}   (mong doi: (0,2) - chan X thang hang 0)")
```

```
STATE #1 (luot X):
  0 1 2
0 X X .
1 O O .
2 . . .

Minimax chon: (0, 2)   (mong doi: (0,2) - X thang ngay hang 0)

STATE #2 (luot O):
  0 1 2
0 X X .
1 . O .
2 . . .

Minimax chon: (0, 2)   (mong doi: (0,2) - chan X thang hang 0)
```

Câu 3b – Tính tay state #1 (lượt X):

Bảng:

```
X | X | _
O | O | _
_ | _ | _
```

Ô trống: `(0,2), (1,2), (2,0), (2,1), (2,2)`

Phân tích từng nước đi của X:

- **X đi `(0,2)`** → hàng 0 = `X,X,X` → **X thắng ngay** → utility = 1
- **X đi `(1,2)`** → chặn O (vì O đe dọa hàng 1). Game tiếp tục, kết quả = 0
- **X đi `(2,0)`, `(2,1)`, hoặc `(2,2)`** → KHÔNG chặn O → O sẽ đi `(1,2)` → hàng 1 = `O,O,O` → **O thắng** → utility = -1

X là MAX → chọn nước có `min_value` LỚN nhất.
Kết luận: Minimax chọn ô **(0,2)**

Câu 3c – Tính tay state #2 (lượt O):

Bảng:

```
X | X | _
_ | O | _
_ | _ | _
```

X đang đe dọa thắng tại ô **(0,2)** (X có 2 quân liên tiếp ở hàng 0).

Nếu O **không chặn**, lượt sau X sẽ thắng → utility = +1 → tệ cho O.
Vậy O (MIN) phải chọn ô để **chặn** X.

Kết luận: Minimax (O) chọn ô **(0,2)**

```
# CAU 3d – Cai dat vong lap game (nguoi vs AI)
# ============================================================

def play_game():
    """
    Vong lap game: Nguoi (X) vs AI (O).
    - Nguoi nhap toa do dang 'hang cot' (vd: '1 2')
    - AI dung minimax() chon nuoc di toi uu
    - In bang sau moi luot, kiem tra ket thuc
    """
    board = initial_state()
    print("=" * 40)
    print("TICTACTOE – Nguoi (X) vs AI (O)")
    print("Nhap toa do dang: hang cot (vd: 1 2)")
    print("=" * 40)
    print_board(board)

    while not terminal(board):
        current = player(board)
        if current == X:
            # TODO: Nhan input nguoi choi, parse 'hang cot', validate (trong bang & o trong)
            #       Neu sai -> yeu cau nhap lai
            #       Neu dung -> board = result(board, action)
            while True:
                try:
                    move_str = input(f"Lượt của {current}. Nhập nước đi (hàng cột, vd: 1 2): ")
                    row, col = map(int, move_str.split())
                    action = (row, col)

                    if not (0 <= row < 3 and 0 <= col < 3):
                        print("Ô không hợp lệ. Hãy nhập hàng và cột từ 0 đến 2.")
                    elif action not in actions(board):
                        print("Ô đã có người đánh hoặc không hợp lệ. Hãy chọn ô khác.")
                    else:
                        board = result(board, action)
```

```
                        break
                except ValueError:
                    print("Đầu vào không hợp lệ. Hãy nhập hai số cách nhau bởi dấu cách (vd: 1 2).")
            pass
        else:
            # TODO: AI dung minimax()
            #       In thong bao 'AI di o (i,j)'
            #       board = result(board, action)
            print(f"Lượt của {current} (AI). Đang tính toán nước đi...")
            action = minimax(board)
            board = result(board, action)
            print(f"AI đi vào ô {action}")
            pass
        print_board(board)

    # TODO: In ket qua: 'X thang!' / 'O thang!' / 'Hoa!'
    final_winner = winner(board)
    if final_winner == X:
        print("X THẮNG!")
    elif final_winner == O:
        print("O THẮNG!")
    else:
        print("HÒA!")

    pass


# Bo comment dong duoi de chay (tat khi nop bai)
# play_game()
```

```
========================================
TICTACTOE – Nguoi (X) vs AI (O)
Nhap toa do dang: hang cot (vd: 1 2)
========================================
  0 1 2
0 . . .
1 . . .
2 . . .

Lượt của X. Nhập nước đi (hàng cột, vd: 1 2): 1 2
  0 1 2
0 . . .
1 . . X
2 . . .

Lượt của O (AI). Đang tính toán nước đi...
AI đi vào ô (1, 1)
  0 1 2
0 . . .
1 . O X
2 . . .

Lượt của X. Nhập nước đi (hàng cột, vd: 1 2): 3 4
Ô không hợp lệ. Hãy nhập hàng và cột từ 0 đến 2.
Lượt của X. Nhập nước đi (hàng cột, vd: 1 2): 0 0\
Đầu vào không hợp lệ. Hãy nhập hai số cách nhau bởi dấu cách (vd: 1 2).
Lượt của X. Nhập nước đi (hàng cột, vd: 1 2): 0 0
  0 1 2
0 X . .
1 . O X
2 . . .

Lượt của O (AI). Đang tính toán nước đi...
AI đi vào ô (0, 1)
  0 1 2
0 X O .
1 . O X
2 . . .

Lượt của X. Nhập nước đi (hàng cột, vd: 1 2): 0 2
  0 1 2
0 X O X
1 . O X
2 . . .
```

```
Lượt của O (AI). Đang tính toán nước đi...
AI đi vào ô (2, 1)
  0 1 2
0 X O X
1 . O X
2 . O .

O THẮNG!
```

Câu 3e – Tại sao Minimax chậm và đề xuất cải tiến:

(Viết ở đây — nhắc đến branching factor, độ sâu cây, và đề xuất một kỹ thuật cụ thể) Minimax chậm chủ yếu do phải duyệt toàn bộ cây trò chơi với branching factor (b) là số nước đi có thể tại mỗi trạng thái và độ sâu cây (d) là số bước cần xét đến trạng thái kết thúc. Độ phức tạp của thuật toán vào khoảng O(b d), nên khi b và d tăng thì số trạng thái cần duyệt tăng theo cấp số mũ, khiến thời gian xử lý trở nên rất lớn. Để cải thiện, một kỹ thuật hiệu quả là Alpha-Beta Pruning, giúp cắt bỏ những nhánh không thể ảnh hưởng đến kết quả cuối cùng. Nhờ đó, số lượng nút cần xét giảm đáng kể (trong trường hợp tốt có thể xuống gần O(b d/2)), giúp Minimax chạy nhanh hơn mà vẫn đảm bảo cho ra quyết định tối ưu.

```
# ============================================================
# CELL NOP BAI – Chay sau khi hoan thanh toan bo de
# Sinh vien KHONG sua cell nay
# ============================================================
import os, subprocess

try:
    from google.colab import drive
    IS_COLAB = True
except ImportError:
    IS_COLAB = False

NOTEBOOK_NAME = "De_Thi_Lan3.ipynb"

print("Buoc 1: Dang chuyen doi sang PDF...")

result = subprocess.run(
    ["jupyter", "nbconvert", "--to", "webpdf",
     "--allow-chromium-download", "--no-input",
     NOTEBOOK_NAME],
    capture_output=True, text=True
)

pdf_file = NOTEBOOK_NAME.replace(".ipynb", ".pdf")

if result.returncode != 0 or not os.path.exists(pdf_file):
    print("  webpdf that bai, chuyen sang HTML...")
    subprocess.run(
        ["jupyter", "nbconvert", "--to", "html", "--no-input", NOTEBOOK_NAME],
        capture_output=True
    )
    html_file = NOTEBOOK_NAME.replace(".ipynb", ".html")
    if os.path.exists(html_file):
        if IS_COLAB:
            from google.colab import files
            files.download(html_file)
        print(f"\n[HTML] File '{html_file}' da tai xuong.")
        print("=> Mo file HTML trong trinh duyet -> Ctrl+P -> Save as PDF")
    else:
        print("Loi: Khong the chuyen doi. Hay dung File > Print trong Colab.")
else:
    print(f"  Thanh cong! File: {pdf_file}")
    if IS_COLAB:
        from google.colab import files
        files.download(pdf_file)
        print(f"\n[PDF] '{pdf_file}' da tai xuong may tinh.")
    else:
        print(f"\n[PDF] Da luu tai: {os.path.abspath(pdf_file)}")
```

```
print("\nDat ten lai file theo dinh dang: MSSV_HoTen.pdf")
print("Nop qua link giao vien da cung cap.")
```

NỘP BÀI

> **Sinh viên chạy cell bên trên để xuất PDF và nộp bài**